The Commons

A Mathematical Framework for Sovereign Resource Allocation

Abstract

We present a mathematical framework for resource allocation based on the Entity-Attribute-Value (EAV) paradigm with **priority aligned capacity distribution**. The system consists of entities with attributes, where special attributes create relational structure (membership), flow gradients (potentials), and allocation constraints. Each attribute has an associated observer who recognized it, creating a **distributed epistemic property attribution system**. We derive a Constrained Weighted Allocation Algorithm and prove that when weights represent net-benefit gradients toward goal realization, honest preference signaling is the unique optimal strategy. This makes strategic manipulation, vote trading, and Sybil attacks strictly suboptimal. The framework unifies categorization, allocation, and type hierarchies within a single structure, providing mathematical foundations for sovereign coordination systems.

1 The Entity-Attribute-Value Model

Definition 1 (The Commons)**.** *Let $\mathbb{E}$ be the universe of entities, $\mathbb{A}$ the universe of attributes, $\mathbb{V}$ the universe of values, $\mathbb{T}$ the universe of types, $\mathbb{O} \subseteq \mathbb{E}$ the set of observers, and $\mathbb{T}_{discrete}$ discrete time. The **Commons** is the quadruple:*

$$\mathcal{C} = \langle \mathcal{N}, \Pi, \Omega, \{\Delta_t\}_{t \in \mathbb{T}_{discrete}} \rangle$$

where:

- $\mathcal{N} \subseteq \mathbb{E}$ *is a finite set of entities*
- $\Pi : \mathcal{N} \to (\mathbb{A} \to \mathbb{V})$ *assigns each entity a mapping from attributes to values*
- $\Omega : \mathcal{N} \times \mathbb{A} \to \mathbb{O}$ *records which observer recognized each attribute*
- $\{\Delta_t\}_{t \in \mathbb{T}_{discrete}}$ *is the allocation history*
- $\mathbb{T}$ *is the universe of types, where $\mathcal{N} \subseteq \mathbb{T}$ (entities can serve as types)*

Remark 1 (Distributed Epistemic Property Attribution)**.** *The Commons uses a **distributed Entity-Attribute-Value (EAV) model with observer recognition**:*

- ***Entities*** *are the fundamental objects (nodes in traditional graph terminology)*
- ***Attributes*** *are property keys (e.g., "memberOf", "potentials", "reputation")*
- ***Values*** *are property values (sets of entities, lists of potentials, numbers, etc.)*

- ***Observers*** *are entities who recognize/assert attributes of other entities*

The observer function Ω *creates a* ***distributed epistemic system****:*

$$\Omega(e, a) = o \quad \textit{means} \quad \textit{"Observer } o \textit{ asserts that entity } e \textit{ has attribute } a\textit{"}$$

This enables:

- ***Perspective****: Different observers may recognize different attributes*
- ***Provenance****: Every attribute has a source (who recognized it)*
- ***Distribution****: No single authority on truth*
- ***Disagreement****: Multiple observers can have conflicting views*

This model is more general than traditional EAV, as it captures who knows what about whom*.*

1.1 Attribute Types

Remark 2 (Value Flexibility)**.** *Values in* $\mathbb{V}$ *can be of any type: primitives (text, numbers, booleans), structured data (lists, maps, records), or domain-specific types (potentials, entity references, etc.). The model imposes no restrictions on value types, allowing maximum flexibility for different use cases.*

Definition 2 (Attribute Recognition)**.** *An* ***attribute recognition*** *(or* ***assertion****) is a tuple:*

$$(o, e, a, v) \in \mathbb{O} \times \mathcal{N} \times \mathbb{A} \times \mathbb{V}$$

meaning "Observer o *recognizes that entity* e *has attribute* a *with value* v*".*

The system state is the set of all current recognitions:

$$\mathcal{R} = \{(o, e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

Remark 3 (Observer-Relative Truth)**.** *In this model, there is no objective truth about attributes — only* ***observer-relative assertions****. An entity may have different attribute values according to different observers:*

$$\Omega(e, a) = o_1 \wedge \Pi(e)(a) = v_1 \quad \textit{(according to } o_1\textit{)}$$

$$\Omega(e, a) = o_2 \wedge \Pi(e)(a) = v_2 \quad \textit{(according to } o_2\textit{)}$$

where $v_1 \neq v_2$ *represents disagreement.*

For implementation, we typically maintain a single "current view" per entity-attribute pair, with Ω *tracking the most recent or most trusted observer.*

1.2 The Membership Attribute

Definition 3 (Membership as Attribute)**.** *The* ***memberOf*** *attribute creates the relational structure. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“}memberOf\text{”}) \in \mathbb{V}_{refs}$$

The membership relation E is ***emergent*** *from this attribute:*

$$E = \{(e_1, e_2) \in \mathcal{N} \times \mathcal{N} \mid e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})\}$$

For $e_1, e_2 \in \mathcal{N}$, we write $e_1 \to e_2$ when $e_2 \in \Pi(e_1)(\text{“}memberOf\text{”})$, meaning “$e_1$ is a member of e_2”.

Definition 4 (Categories and Participants)**.** *For any entity $e \in \mathcal{N}$:*

$$Categories(e) = \Pi(e)(\text{“}memberOf\text{”}) \tag{1}$$

$$Participants(c) = \{e \in \mathcal{N} \mid c \in \Pi(e)(\text{“}memberOf\text{”})\} \tag{2}$$

Definition 5 (Transitive Closure)**.** *Let E^* be the reflexive transitive closure of the emergent relation E. Define:*

$$Ancestors(e) = \{a \in \mathcal{N} \mid (e, a) \in E^* \wedge e \neq a\} \tag{3}$$

$$Descendants(c) = \{d \in \mathcal{N} \mid (d, c) \in E^* \wedge d \neq c\} \tag{4}$$

These capture the full inheritance hierarchy.

Remark 4 (Graph as Emergent Structure)**.** *In the EAV model, the graph $G = \langle \mathcal{N}, E \rangle$ is not a primitive but an* ***emergent structure*** *derived from the memberOf attribute. This has profound implications:*

- *The relational structure is just another attribute, not special*
- *Entities can have arbitrary attributes beyond membership*
- *The model generalizes naturally to other relational patterns*
- *Constraints can uniformly reference any attribute*
- *Different observers may recognize different membership relations*

1.3 Observer-Based Queries

Definition 6 (Observer Query)**.** *For observer $o \in \mathbb{O}$, entity $e \in \mathcal{N}$, and attribute $a \in \mathbb{A}$, define:*

$$\Pi_o(e, a) = \begin{cases} \Pi(e)(a) & \text{if } \Omega(e, a) = o \\ \perp & \text{otherwise} \end{cases}$$

This returns the attribute value only if *observer o recognized it.*

Definition 7 (Observer Assertions)**.** *The set of all assertions by observer o is:*

$$Assertions(o) = \{(e, a, v) \mid \Omega(e, a) = o \wedge \Pi(e)(a) = v\}$$

This captures “what does observer o think about the world?”

Definition 8 (Entity Observers)**.** *The set of observers who have recognized attributes of entity e is:*

$$Observers(e) = \{o \in \mathbb{O} \mid \exists a \in \mathbb{A} : \Omega(e, a) = o\}$$

This captures "who has opinions about entity e?"

Remark 5 (Consensus and Conflict)**.** *When multiple observers have recognized the same attribute of an entity, we may need to aggregate their views. Common strategies include:*

- ***Most recent**: Use the observation with highest timestamp*
- ***Highest confidence**: Weight by observer confidence scores*
- ***Trust-weighted**: Weight by observer reputation/trust*
- ***Consensus**: Use majority value (for discrete attributes)*

For the purposes of allocation, we typically use a single "current view" per entity-attribute pair, with Ω tracking the authoritative observer for that view.

1.4 The Potentials Attribute

Definition 9 (Potential)**.** *A **potential** is a 4-tuple $p = (\tau, q, C, M)$ where:*

- *$\tau \in \mathbb{T}$ is the type (from the universe of types)*
- *$q \in \mathbb{R}^{\pm}$ is the signed magnitude*
- *$C \in \mathcal{C}$ is a constraint predicate*
- *M is metadata (a map of key-value pairs)*

*We say p is a **source** if $q > 0$ and a **sink** if $q < 0$.*

Since $\mathcal{N} \subseteq \mathbb{T}$, entities can serve as types (types-as-nodes), but types can also be abstract identifiers, strings, or other type representations not necessarily corresponding to entities in $\mathcal{N}$.

For a potential $p = (\tau, q, C, M)$, we use projection notation:

- *$\tau(p)$ denotes the type component*
- *$q(p)$ denotes the magnitude component*
- *$C(p)$ denotes the constraint component*
- *$M(p)$ denotes the metadata component*

***Metadata for Space-Time Matching**: The metadata M enables sophisticated matching beyond type compatibility. Common metadata includes:*

- ***Temporal**: time_zone, recurrence (daily/weekly/monthly/yearly), start_date, end_date, availability_window (hierarchical time patterns)*
- ***Spatial**: location_type, coordinates (latitude/longitude), city, country, online_link*

- ***Constraints***: *advance_notice_hours, booking_window_hours, mutual_agreement_required*
- ***Descriptive***: *name, unit, description*

Two potentials p_1 (source) and p_2 (sink) are ***compatible*** *if:*

1. *Type matches:* $\tau(p_1) = \tau(p_2)$
2. *Constraint satisfied:* $C(p_2)$ *accepts the source entity*
3. *Metadata compatible: A user-provided match function* $\phi(M(p_1), M(p_2))$ *returns true*

The match function ϕ can check arbitrary compatibility criteria, such as:

- *Time overlap (accounting for timezones and recurrence patterns)*
- *Location proximity or online availability*
- *Mutual agreement requirements*

Terminology: *For intuitive exposition, we refer to a potential's magnitude as its* ***capacity*** *when the potential is a source ($q > 0$) and as its* ***need*** *when the potential is a sink ($|q|$ where $q < 0$).*

Definition 10 (Potentials as Attribute)**.** *The* ***potentials*** *attribute stores flow gradients. For entity $e \in \mathcal{N}$:*

$$\Pi(e)(\text{“potentials”}) \in \mathbb{V}_{pots}$$

This is a list of potentials representing the entity's capacities (sources) and needs (sinks).

Remark 6 (Types from Universe $\mathbb{T}$)**.** *In this framework, types come from the universe $\mathbb{T}$, which includes entities as a special case ($\mathcal{N} \subseteq \mathbb{T}$). This means:*

Types can be entities *(types-as-nodes): When $\tau \in \mathcal{N}$, the type is an entity that may itself have:*

- *Categories (via memberOf attribute, enabling type hierarchies)*
- *Potentials (via potentials attribute, enabling meta-properties and flows)*
- *Participants (entities with this entity in their memberOf)*

Types can be abstract: *When $\tau \in \mathbb{T} \setminus \mathcal{N}$, the type is an abstract identifier (e.g., string, UUID) not corresponding to any entity in the system.*

This creates a flexible type system: use types-as-nodes when you want rich type metadata and hierarchies, use abstract types when you just need simple type matching. For example, an entity representing “MealType” could have “Food” and “Consumable” in its memberOf attribute, while also being referenced as the type in potentials like $(\text{MealType}, +100)$.

Definition 11 (Accessing Potentials)**.** *For entity $e \in \mathcal{N}$, we define:*

$$P(e) = \Pi(e)(\text{“potentials”})$$

This is the list of potentials for entity e. We can filter by type:

$$P(e, \tau) = \{p \in P(e) \mid \tau(p) = \tau\}$$

1.5 Constraints and Weights

Definition 12 (Constraint)**.** *A **constraint** is a function $C : \mathcal{N} \to \mathbb{R}_{\geq 0} \cup \{\infty\}$ that limits the maximum flow to each entity:*

- $C(e) = 0$ *excludes e from receiving flow*
- $C(e) = \infty$ *imposes no limit on e*
- $C(e) = k \in (0, \infty)$ *caps flow to e at k*

Constraints express policy limits, rate caps, distance bounds, and other restrictions on flow.

***Attribute-based constraints**: Constraints can reference any attribute:*

$$C_{reputation}(e) = \begin{cases} \infty & \text{if } \Pi(e)(\text{“reputation”}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Definition 13 (Filter as Special Constraint)**.** *A **filter** is a constraint that takes only values in $\{0, \infty\}$:*

$$\Phi : \mathcal{N} \to \{0, \infty\} \subset \mathbb{R}_{\geq 0} \cup \{\infty\}$$

Filters determine binary eligibility (include/exclude) and are a special case of constraints.

Definition 14 (Weight)**.** *A **weight function** is $w : \mathcal{N} \to \mathbb{R}_{\geq 0}$ assigning preference to each entity.*

***Attribute-based weights**: Weights can reference any attribute:*

$$w_{reputation}(e) = \Pi(e)(\text{“reputation”})$$

1.6 Allocation Records

Definition 15 (Record)**.** *An **allocation record** at time t is:*

$$\Delta_t = (t, s, \tau, \{(r_i, q_i)\}_{i=1}^{n}, C, w, M)$$

where s is the source entity, τ the type, $\{(r_i, q_i)\}$ the recipient-quantity pairs, and M metadata.

2 Flow Equations: From Simple to Complete

2.1 Constrained Weighted Allocation

Remark 7 (Sovereign Coordination Requires Dual Priorities)**.** *In sovereign coordination systems, both providers and recipients have autonomy over their preferences. A provider chooses how to distribute their capacity among recipients based on their priorities. Simultaneously, a recipient chooses which providers they wish to receive from based on trust, alignment, quality, or other factors.*

*The allocation mechanism must respect **both** parties' preferences via a two-phase process:*

1. ***Phase 1 (Provider Constraints)**: Allocates capacity based on provider priorities, subject to need and capacity limits.*
2. ***Phase 2 (Recipient Refinement)**: Allows recipients to adjust their source mix based on their own priorities, without violating Phase 1 limits.*

2.2 Constrained Weighted Allocation

Theorem 1 (Phase 1: Constrained Weighted Allocation)**.** *Let $s \in \mathcal{N}$ be a source entity with capacity C. Let R be the set of eligible recipients, where each $r \in R$ has declared need N_r. Let $w_s(r)$ be the provider's weight for recipient r, normalized such that $\sum_{r \in R} w_s(r) = 1$.*

The allocations $\{A_r\}$ are the solution to the convex optimization problem:

$$\textit{Minimize} \sum_{r \in R} (A_r - w_s(r) \cdot C)^2$$

Subject to constraints:

1. $0 \leq A_r \leq N_r$ *(Need Constraint)*
2. $\sum_{r \in R} A_r \leq C$ *(Capacity Constraint)*

This minimizes the deviation from the provider's ideal weighted distribution while respecting strict physical constraints.

Proof. The objective function is strictly convex (sum of squares) and the feasible region defined by the linear constraints is convex. Therefore, a unique global minimum exists. The solution A_r represents the closest possible allocation to the target $w_s(r) \cdot C$ that satisfies all needs and capacity limits. □ □

Theorem 2 (Phase 2: Recipient Source Refinement)**.** *Let r be a recipient receiving total allocation $A_r^{total} = \sum_{s \in S} A_{sr}$ from a set of sources S. Let $w_r(s)$ be the recipient's target weight for source s (percent of need to be filled by s).*

The recipient seeks to adjust strictly positive flows $\{A_{sr}\}$ to minimize:

$$\textit{Minimize} \sum_{s \in S} \left(\frac{A_{sr}}{A_r^{total}} - w_r(s) \right)^2$$

subject to $\sum_s A_{sr} = A_r^{total}$ and Phase 1 constraints ($0 \leq A_{sr} \leq C_{sr}$).

Remark 8 (Satisfaction Over Composition)**.** *Phase 2 is a preference refinement. If exact composition matching reduces total flow (e.g. insisting on 50% from a provider with zero capacity), the system prioritizes **satisfaction** (meeting needs) over **composition**. Source preferences are optimization targets, not hard constraints that block flow.*

2.3 Optimization Properties

Proposition 1 (Existence and Uniqueness)**.** *The objective function in Phase 1 (Theorem **??**) is strictly convex, and the constraints form a compact convex set. Thus, a unique optimal allocation A^* exists.*

Proposition 2 (Pareto Efficiency)**.** *The solution is Pareto efficient with respect to the weighted preferences. No allocation can be increased without violating a capacity constraint or a need constraint, or increasing the deviation from the provider's weighted preference.*

Proposition 3 (Full Utilization)**.** *If total need exceeds capacity, the optimization ensures $\sum A_r = C$. Capacity is never wasted if there is unmet need that can be satisfied.*

Proposition 4 (Universal Satisfaction in Sparse Networks)**.** *If total need is less than capacity ($\sum_{r \in R} N_r \leq C$), the optimization allocates $A_r = N_r$ for all r. The system automatically achieves universal need satisfaction, distributing surplus capacity only to those with unmet need.*

Proof. Since $\sum N_r \leq C$, the allocation $A_r = N_r$ is feasible. Any allocation $A_r < N_r$ would leave unmet need and unused capacity (or reduced fit to weights). The optimization objective, combined with the capacity constraint, ensures that capacity effectively "flows" to fill all accessible needs before being conserved. □ □

3 Distributed Coordination

Remark 9 (Phase 2 as Coordination)**.** *The recipient source refinement process (Theorem **??**) inherently solves the multi-provider coordination problem. By adjusting their source preferences $w_r(s)$ and broadcasting them to providers, recipients guide independent providers toward a globally consistent allocation.*

Proposition 5 (Convergence)**.** *Since the Phase 1 optimization (Provider Constraints) is convex and Phase 2 (Refinement) is a convex adjustment on the simplex, the coupled system converges to the global optimum of the joint preference function.*

4 Aggregation Along Edges

Definition 16 (Upward Aggregation)**.** *For entity e and type τ:*

$$Agg_{\uparrow}(e, \tau) = \sum_{c \in Categories(e)} \sum_{\substack{p \in P(c) \\ \tau(p) = \tau}} q(p)$$

Definition 17 (Downward Aggregation)**.** *For category c and type τ:*

$$Agg_{\downarrow}(c, \tau) = \sum_{e \in Participants(c)} \sum_{\substack{p \in P(e) \\ \tau(p) = \tau}} q(p)$$

Proposition 6 (Aggregation Duality)**.** *Aggregation preserves the sign structure:*

$$Agg_{\uparrow}(n, \tau) \in \mathbb{R}^{\pm} \tag{5}$$

$$Agg_{\downarrow}(c, \tau) \in \mathbb{R}^{\pm} \tag{6}$$

Proof. Both aggregations are sums of signed quantities $q(p) \in \mathbb{R}^{\pm}$. Sums of signed reals remain in $\mathbb{R}^{\pm}$. □ □

5 Emergent Properties

Theorem 3 (Entity-Type Duality)**.** *Every entity $e \in \mathcal{N}$ can simultaneously be:*

1. *A **type** if Participants$(e) \neq \emptyset$*

2. *An* ***instance*** *if Categories*$(e) \neq \emptyset$ *(i.e.,* $\Pi(e)($*"memberOf"*$) \neq \emptyset$*)*

3. *A* ***concrete entity*** *if* $P(e) \neq \emptyset$ *(i.e.,* $\Pi(e)($*"potentials"*$) \neq \emptyset$*)*

4. *A* ***potential type*** *if* $\exists p \in P(e') : \tau(p) = e$ *for some* $e' \in \mathcal{N}$

These roles are not mutually exclusive. Since types are entities ($\tau \in \mathcal{N}$), an entity referenced as a type in a potential can simultaneously be an instance of other types, creating rich type hierarchies.

Proof. The EAV structure makes no distinction between entities serving different roles. An entity may have:

- Other entities with it in their memberOf attribute (making it a type)
- Entities in its own memberOf attribute (making it an instance)
- Values in its potentials attribute (making it concrete)
- Be referenced as τ in other entities' potentials (making it a potential type)

All four roles can coexist. □ □

Theorem 4 (Non-Accumulation)**.** *No node can accumulate beyond its need. Formally, if r receives flow f of type τ and has need $|q_r|$ (where $q_r < 0$), then:*

$$f \leq |q_r|$$

Proof. Immediate from the min function in the Unified Flow Equation. □ □

Theorem 5 (Proportional Fairness)**.** *The allocation ratios equal the weight ratios. For recipients $r_1, r_2 \in R$:*

$$\frac{Flow(s, r_1, \tau)}{Flow(s, r_2, \tau)} = \frac{w(r_1)}{w(r_2)}$$

when neither is need-constrained.

Proof. When $\text{Flow}(s, r_i, \tau) = q_s \cdot \frac{w(r_i)}{\sum_{r' \in R} w(r')}$ for $i = 1, 2$ (i.e., neither hits the min bound), the ratio simplifies to $\frac{w(r_1)}{w(r_2)}$. □ □

Theorem 6 (Determinism)**.** *The allocation function is deterministic: identical inputs produce identical outputs. Formally:*

$$\forall S_1, S_2 : \textit{if } (G_1, P_1, C_1, w_1) = (G_2, P_2, C_2, w_2) \textit{ then } Flow_1 = Flow_2$$

Proof. The Unified Flow Equation is a pure mathematical function with no randomness or hidden state. Each step (constraint checking, weight computation, normalization, allocation) is deterministic. Therefore, identical system states produce identical flows. □ □

6 The Three-Layer Ontology

Definition 18 (Ontological Layers)**.** *The Commons embodies three layers of reality:*

1. ***Structural Layer**: Entity-attribute structure defines what exists through relationships*
2. ***Potential Layer**: Potentials attribute defines what* could *flow through gradients*
3. ***Actual Layer**: Records $\{\Delta_t\}$ define what* did *flow through events*

Remark 10 (Ontological Priority)**.** *The layers exhibit a natural precedence:*

$$\mathcal{N}, \Pi \prec \textit{potentials} \prec \{\Delta_t\}$$

where $\prec$ denotes ontological precedence: entities and their attributes must exist before having potentials, and potentials must exist before being actualized. This is a design principle rather than a mathematical theorem.

7 Composition of Constraints and Weights

Definition 19 (Constraint Algebra)**.** *Constraints form a semilattice under minimum:*

$$(C_1 \wedge C_2)(n) = \min(C_1(n), C_2(n)) \tag{7}$$
$$\top(n) = \infty \quad \textit{(no constraint)} \tag{8}$$
$$\bot(n) = 0 \quad \textit{(total exclusion)} \tag{9}$$

The minimum operation selects the tightest constraint.

Proposition 7 (Filter Algebra as Special Case)**.** *When constraints are restricted to $\{0, \infty\}$, they form a Boolean algebra:*

$$(\Phi_1 \wedge \Phi_2)(n) = \min(\Phi_1(n), \Phi_2(n)) \tag{10}$$
$$(\Phi_1 \vee \Phi_2)(n) = \max(\Phi_1(n), \Phi_2(n)) \tag{11}$$
$$(\neg\Phi)(n) = \begin{cases} \infty & \textit{if } \Phi(n) = 0 \\ 0 & \textit{if } \Phi(n) = \infty \end{cases} \tag{12}$$

This recovers standard Boolean filter composition.

Definition 20 (Constraint Examples)**.** *Common constraint patterns include:*

$$C_{\textit{filter}}(n) = \begin{cases} \infty & \textit{if eligible}(n) \\ 0 & \textit{otherwise} \end{cases} \quad \textit{(binary)} \tag{13}$$
$$C_{\textit{cap}}(n) = k \quad \textit{(absolute limit)} \tag{14}$$
$$C_{\textit{percent}}(n) = \alpha \cdot q_s \quad \textit{(proportional limit)} \tag{15}$$
$$C_{\textit{distance}}(n) = \max(0, d_{\max} - d(s, n)) \quad \textit{(spatial decay)} \tag{16}$$
$$C_{\textit{rate}}(n, t) = r \cdot \Delta t \quad \textit{(temporal limit)} \tag{17}$$

Definition 21 (Weight Function)**.** *A **weight function** is $w : N \to \mathbb{R}_{\geq 0}$ assigning preference to each node, subject to normalization:*

$$\sum_{r \in N} w(r) = 1$$

Proposition 8 (Weight Algebra)**.** *Weights form a semiring under:*

$$(w_1 \oplus w_2)(n) = w_1(n) + w_2(n) \tag{18}$$
$$(w_1 \otimes w_2)(n) = w_1(n) \cdot w_2(n) \tag{19}$$

Note that $\otimes$ preserves proportions but requires renormalization: $w' = (w_1 \otimes w_2)/\sum_r (w_1 \otimes w_2)(r)$.

Definition 22 (Weight Examples)**.** *Common weight patterns include:*

***Uniform weights**: $w_{uniform}(n) = 1/|N|$ for all $n \in N$*

***Priority Alignment**: Each node n assigns priority weights $w_n : N \to [0, 1]$ with $\sum_r w_n(r) = 1$. The **alignment** between v_1 and v_2 is:*

$$Align(v_1, v_2) = \min(w_{v_1}(v_2), w_{v_2}(v_1))$$

Normalized alignment values into weights:

$$w_{Align}(n) = \frac{Align(p, n)}{\sum_{r \in N} Align(p, r)}$$

***Composite weights**: Combine multiple factors through multiplication and renormalization:*

$$w_{composite}(n) \propto w_1(n) \cdot w_2(n) \cdot \ldots \cdot w_k(n)$$

For example, social allocation might weight by priority alignment and reputation:

$$w_{social}(n) = \frac{Align(provider, n) \cdot reputation(n)}{\sum_{r \in N} Align(provider, r) \cdot reputation(r)}$$

where reputation could be derived from past allocation records or other sources.

Proposition 9 (Composability)**.** *Complex allocation strategies emerge from composition:*

$$C_{complex} = C_1 \wedge C_2 \wedge \cdots \wedge C_n \quad \textit{(tightest constraint)} \tag{20}$$
$$w_{complex} \propto w_1 \otimes w_2 \otimes \cdots \otimes w_n \quad \textit{(multiplicative preference)} \tag{21}$$

8 Dynamic Equilibrium

Definition 23 (Fill Fraction)**.** *For a given allocation round, the **fill fraction** is:*

$$f = \frac{\sum_{r \in N} \sum_{\tau} Flow(s, r, \tau)}{\sum_{r \in N} \sum_{p \in P(r), q(p) < 0} |q(p)|}$$

This measures what fraction of total need is satisfied per round.

Theorem 7 (Convergence to Equilibrium)**.** *Let* $\mathcal{S}^+ = \sum_{s \in N} \sum_{p \in P(s), q(p)>0} q(p)$ *be total capacity and* $\mathcal{S}^- = \sum_{r \in N} \sum_{p \in P(r), q(p)<0} |q(p)|$ *be total need.*

Static Case: *If* $\mathcal{S}^+ \geq \mathcal{S}^-$ *and network state is fixed, then:*

$$\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$$

where $\|r(t)\|$ *is the total unsatisfied need at time* t*. The system converges to full satisfaction in:*

$$T_\varepsilon = O\left(\frac{\log(1/\varepsilon)}{\log(1/(1-f))}\right) \text{ rounds}$$

Dynamic Case: *When needs and capacities evolve over time, the system does not converge to a fixed point but instead recomputes optimal allocation at each time step based on the current state* $S(t) = (G^{(t)}, P^{(t)}, C^{(t)}, w^{(t)})$*. The allocation at time* t *is optimal for the state at time* t*, providing adaptive rather than convergent behavior.*

Proof. **Static case:** Each round satisfies fraction f of remaining needs. By the allocation capping property, receiving always reduces need. Thus:

$$\|r(t+1)\| = \|r(t) - \phi(r(t))\| \leq (1-f) \cdot \|r(t)\|$$

Iterating gives $\|r(t)\| \leq (1-f)^t \cdot \|r(0)\|$. Setting $(1-f)^t \cdot \|r(0)\| \leq \varepsilon$ and solving for t yields the convergence rate.

Dynamic case: The system recomputes optimal allocation for current state $S(t)$ at each step, maintaining instantaneous optimality without requiring convergence to zero. □ □

Definition 24 (Damping for Stability)**.** *To prevent oscillations in allocation, introduce a damping factor* $\delta(r) \in \{0.5, 0.8, 1.0\}$ *based on oscillation detection. The* ***active need*** *becomes:*

$$q_{active}(r) = q_{declared}(r) \times \delta(r)$$

where $\delta(r)$ *decreases when oscillations are detected in* r*'s allocation history.*

Proposition 10 (Damping Preserves Convergence)**.** *Damping factors* $\delta \in (0, 1]$ *reduce the fill fraction but preserve the convergence property:*

$$\|r(t)\| \leq (1-f')^t \cdot \|r(0)\|$$

where $f' = f \cdot \min_r \delta(r) > 0$*. The system still converges, though potentially at a slower rate.*

Proof. Damping reduces active needs to $q_{\text{active}}(r) = \delta(r) \cdot q_{\text{declared}}(r)$. The fill fraction becomes $f' = f \cdot \min_r \delta(r)$ since less need is expressed. Since $\delta(r) \in (0, 1]$, we have $f' > 0$, and the convergence argument from Theorem 13 applies with f' replacing f. □ □

9 Sovereignty and Anti-Gaming

Remark 11 (Why Normalization Matters)**.** *Weight normalization* ($\sum_{r \in N} w(r) = 1$) *is essential for meaningful optimization. Without it, entities could assign arbitrarily large weights to all nodes, making* $w(r_1)/\sum_{r'} w(r') = 1/n$ *independent of preferences. Normalization creates scarcity, forcing genuine trade-offs in preference expression.*

9.1 Weak Monotonicity

Axiom 1 (Weak Monotonicity)**.** *The allocation function satisfies weak monotonicity in weights:*

$$\frac{\partial Flow(s, r, \tau)}{\partial w(r)} \geq 0$$

Increasing a recipient's weight does not decrease their allocation.

Proposition 11 (Monotonicity as Respect)**.** *Weak monotonicity is not merely technical—it is the mathematical expression of respecting sovereign preference expression.*

If violated*: Increasing weight to beneficial partner r decreases allocation to r, creating perverse incentives to misrepresent preferences.*

When satisfied*: Honest preference expression is optimal—no strategic manipulation needed.*

9.2 The Anti-Gaming Theorem

We now prove that the Commons framework guarantees anti-gaming properties when weights represent benefit gradients.

Definition 25 (Net-Benefit Gradient)**.** *For entity e with goal G_e, define the* ***net-benefit gradient*** *from provider p:*

$$\beta_e(p) = \frac{\partial \mathbb{P}(G_e)}{\partial received(p, e)} \in \mathbb{R}$$

This measures how much receiving from p contributes to achieving e's goal:

- $\beta_e(p) > 0$*: Receiving from p helps achieve G_e (beneficial)*
- $\beta_e(p) = 0$*: Receiving from p is neutral to G_e (irrelevant)*
- $\beta_e(p) < 0$*: Receiving from p harms achievement of G_e (costly)*

Examples*:*

- ***Hospital****: A patient receiving critical care has $\beta_{hospital}(patient) > 0$ because providing care realizes the hospital's goal of "providing critical care". The patient contributes to goal realization by being a recipient.*
- ***Need-based allocation****: An entity distributing food may set $\beta_e(p) \propto urgency(p)$ or $\beta_e(p) \propto need(p)$, prioritizing those who most need it.*
- ***Harmful relationships****: If receiving from p undermines e's goals (e.g., accepting resources with strings attached that conflict with mission), then $\beta_e(p) < 0$.*

This generalizes beyond private benefit to any *goal-directed allocation, including altruistic, mission-driven, or collective goals.*

Theorem 8 (Anti-Gaming)**.** *Suppose entity e controls weights $w_e : N \to \mathbb{R}_{\geq 0}$ subject to normalization $\sum_r w_e(r) = 1$. If:*

1. *Allocation satisfies the Unified Flow Equation*

2. *Weights satisfy weak monotonicity (Axiom* **??***)*

3. *Entity seeks to maximize goal achievement:* $\max_{w_e} \mathbb{P}(G_e)$

Then the optimal weight allocation is:

$$w_e^*(r) \propto \max(0, \beta_e(r))$$

Setting weights proportional to positive *net-benefit gradients is optimal.*

Implication*: Honest signaling of relationship benefit is the best strategy. Entities with* $\beta_e(r) \leq 0$ *should receive zero weight. Gaming (allocating to non-beneficial or harmful partners) is strictly suboptimal.*

Proof. Entity e's goal achievement depends on received allocations:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot \text{Flow}(p, e, \tau)\right)$$

From the Unified Flow Equation:

$$\text{Flow}(p, e, \tau) = \min\left(q_p \cdot \frac{w_e(p)}{\sum_{r \in R} w_e(r)}, |q_e|, C(e)\right)$$

In the unconstrained regime (not hitting need limits):

$$\text{Flow}(p, e, \tau) = q_p \cdot \frac{w_e(p)}{\sum_{r \in R} w_e(r)}$$

Substituting:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot q_p \cdot \frac{w_e(p)}{\sum_r w_e(r)}\right)$$

With normalization $\sum_r w_e(r) = 1$:

$$\mathbb{P}(G_e) = f\left(\sum_{p \in N} \beta_e(p) \cdot q_p \cdot w_e(p)\right)$$

To maximize, take derivative with respect to $w_e(p)$ subject to $\sum_r w_e(r) = 1$:

$$\mathcal{L} = \sum_p \beta_e(p) \cdot q_p \cdot w_e(p) - \lambda\left(\sum_r w_e(r) - 1\right)$$

First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial w_e(p)} = \beta_e(p) \cdot q_p - \lambda = 0$$

Therefore:

$$w_e^*(p) \propto \beta_e(p) \cdot q_p$$

Conclusion: Optimal weights are proportional to the product of net-benefit gradients and provider capacities: $w_e^*(p) \propto \max(0, \beta_e(p) \cdot q_p)$. This naturally accounts for varying provider capacities: larger capacity providers receive proportionally more weight. Allocating weight to neutral partners ($\beta_e(p) = 0$) or harmful partners ($\beta_e(p) < 0$) is strictly suboptimal regardless of their capacity. □ □

Remark 12 (Capacity-Aware Weighting Emerges Naturally)**.** *Recipients don't need explicit knowledge of provider capacities. The iterative convergence process naturally teaches them: providers with larger capacity consistently deliver more, leading recipients to increase their requested flow from those providers over time. This is a form of* ***learned prioritization*** *where the system discovers optimal weights through experience, not through explicit capacity information exchange.*

Corollary 1 (Honest Signaling)**.** *Under the Commons framework:*

- ***Beneficial partners****: Allocate weight proportional to benefit → receive more*
- ***Non-beneficial partners****: Allocate zero weight → receive nothing*
- ***Gaming (false signaling)****: Allocate weight to non-beneficial partners → reduces allocation from beneficial ones → decreases goal achievement*

Honest preference expression is optimal.

Corollary 2 (Sybil Resistance)**.** *Creating fake identities (Sybils) provides no advantage:*

Proof*: If entity e creates Sybil s and allocates weight $w_e(s) > 0$:*

- *Sybil s has zero capacity ($q_s = 0$) or provides no benefit ($\beta_e(s) = 0$)*
- *By anti-gaming theorem, $w_e^*(s) = 0$*
- *Allocating $w_e(s) > 0$ reduces weight available for beneficial partners*
- *Result: Decreases goal achievement*

Sybil creation is strictly suboptimal. □

9.3 Controllability and Memory

Proposition 12 (Controllability)**.** *The Commons framework permits memory in weight functions, provided current weights retain ultimate control.*

Valid (memory with controllability)*:*

$$w^{(t)}(r) = \alpha \cdot w^{(t)}_{current}(r) + \beta \cdot w^{(t-1)}_{past}(r) \quad \text{with } \alpha > 0$$

Current weights can override past patterns.

Invalid (memory without controllability)*:*

$$w^{(t)}(r) = \begin{cases} w^{(t)}_{current}(r) & \text{if never decreased in past} \\ 0 & \text{if ever decreased in past} \end{cases}$$

Past actions create dead zones—violates sovereignty.

Key insight*: Learning, adaptation, and temporal smoothing are permitted as long as current weights can achieve any desired allocation.*

10 The Paradigm Shift

Proposition 13 (Unification)**.** *The Commons framework provides a unified treatment of:*

1. ***Categorization*** *(memberOf attribute creating emergent graph)*
2. ***Resource allocation*** *(potentials attribute enabling flow equilibration)*
3. ***Type inheritance*** *(upward aggregation along memberOf)*
4. ***Computation*** *(constraint satisfaction + preference optimization)*

Each emerges as a pattern within the Entity-Attribute-Value model.

Proof.

- Categorization: memberOf attribute creates type hierarchies (Definition 5)
- Allocation: The Unified Flow Equation distributes potentials (Theorem 1)
- Inheritance: $\text{Agg}_\uparrow$ propagates along memberOf (Definition 11)
- Computation: Constraints C limit possibilities, weights w optimize preferences (Section 6)

Each is expressible within $\mathcal{C} = \langle \mathcal{N}, \Pi, \{\Delta_t\} \rangle$. □ □

11 A Simple Example

Consider a kitchen k with meal potential $(\texttt{meals}, +100)$ and two people: Alice with potential $(\texttt{meals}, -20)$ and Bob with potential $(\texttt{meals}, -30)$. Using unit weights ($w(n) = 1$ for all n), the Unified Flow Equation gives:

$$\text{Flow}(k, \text{Alice}, \texttt{meals}) = \min\left(100 \cdot \frac{1}{1+1}, 20\right) = \min(50, 20) = 20 \tag{22}$$

$$\text{Flow}(k, \text{Bob}, \texttt{meals}) = \min\left(100 \cdot \frac{1}{1+1}, 30\right) = \min(50, 30) = 30 \tag{23}$$

Alice receives 20 meals, Bob receives 30 meals, in exact proportion to the
ir needs. The allocation is automatic, fair, and complete.

12 Conclusion

Five primitives—graphs, potentials, constraints, weights, and records—yield one equation. From this equation emerges a remarkable property: when weights represent benefit gradients, honest preference signaling is the unique optimal strategy. Strategic manipulation, vote trading, coalition formation, and Sybil attacks all become strictly suboptimal.

This provides mathematical foundations for sovereign coordination: systems where participants retain full autonomy over their preferences while achieving collectively optimal resource allocation. Categorization, allocation, and type hierarchies all emerge as patterns within the same unified structure.